

Invited: Automatic Hardware/Software Design for High-Speed Autonomous Unmanned Aerial Vehicles Guided by a Flight Model

Yuanfan Xu
Tsinghua University

Jincheng Yu*
Tsinghua University

Suquan Zhang
Tsinghua University

Yunfei Xiang
Tsinghua University

Hongyang Jia
Tsinghua University

Yu Wang*
Tsinghua University

ABSTRACT

Autonomous Unmanned Aerial Vehicles (UAVs) are on the rise in the industrial and academic communities. Since most UAVs are severely size, weight, and power (SWaP) constrained, building computing system for high-speed UAVs is challenging. Current domain-specific hardware-software (HW-SW) designs for UAVs are mainly bottom-up, focusing on optimizing a single module in the whole system, such as visual-inertial odometry (VIO), depth estimation, or planning. But this leads to underdesign for flight speed as agile navigation depends on a tight combination of multiple modules. To find the optimal HW-SW design for systematic flight performance, we propose a top-down automatic design framework. A flight model is introduced to guide the inter-module and the intra-module HW-SW optimization towards the system-level goal. For the perception algorithms, we define the representative design space. And some critical non-AI operators are accelerated and profiled on embedded GPU to achieve better hardware performance. The design framework is evaluated on a micro UAV equipped with a Nvidia Jetson Orin NX. In a specific navigation scenario, the design found by our framework achieve 40% and 65% increase on flight speed than two manual design methods respectively.

CCS CONCEPTS

• Computer systems organization → Robotics; • Hardware → Hardware-software codesign.

KEYWORDS

Robotics, Hardware-Software Co-Design, Design Automation

ACM Reference Format:

Yuanfan Xu, Jincheng Yu, Suquan Zhang, Yunfei Xiang, Hongyang Jia, and Yu Wang. 2024. Invited: Automatic Hardware/Software Design for High-Speed Autonomous Unmanned Aerial Vehicles Guided by a Flight Model. In *61st ACM/IEEE Design Automation Conference (DAC '24)*, June 23–27, 2024, San Francisco, CA, USA. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3649329.3663516>

*Corresponding authors. Email: yu-jc@tsinghua.edu.cn, yu-wang@tsinghua.edu.cn

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
DAC '24, June 23–27, 2024, San Francisco, CA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0601-1/24/06
<https://doi.org/10.1145/3649329.3663516>

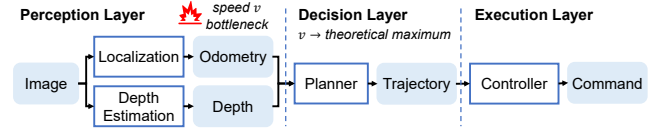


Figure 1: The basic high-speed navigation framework. Conducting SW-HW optimization in Perception Layer is the key to breaking through the speed bottleneck.

1 INTRODUCTION

Over the last decade, autonomous unmanned aerial vehicles (UAVs) have achieved significant development. These agile machines popularized by quadcopters are widely applied in surveillance, exploration, search and rescue, and more.

In all drone applications, high-speed navigation in cluttered and unknown environments is the most fundamental capability of the autonomous system. Thus flight speed naturally becomes the most critical metric for evaluating a UAV system. Autonomous high-speed flight of UAVs relies on a tight coupling of fast and accurate perception with effective planning [4, 8]. Fig. 1 illustrates a basic and general UAV vision-based navigation framework. The planner takes the estimated depth and pose as inputs, and generates a smooth trajectory for the controller to follow.

As the newly proposed planners become more efficient [8, 10], perception becomes the bottleneck limiting further increases in flight speed [2]. For example, the SOTA learning-based planner [8] can reach 12m/s in some scenarios, while the perception capability limits the theoretical maximum safe speed to 13.5m/s. Therefore, designing efficient perception instead of planning modules is more promising in the future.

However, it is challenging to design perception software and hardware on a SWaP constrained UAV for maximizing its safe flight speed. *Firstly, multiple perception components run together to affect flight speed, and optimizing a single component results in a waste of resources.* There are lots of work to design dedicated algorithms or accelerators for a UAV's perception module, such as localization [9, 12]. If we put most computational resources on optimizing localization, it is highly likely that the depth estimation module will perform poorly and become the bottleneck limiting flight speed. *Secondly, perception algorithms are diverse, and traditional optimization methods based on manual experience can only cover a small part of parameter space.* Expert experience always drives us to choose the algorithm parameters with the highest accuracy or lowest latency, but this does not necessarily imply an optimal solution. We need an automatic method to traverse the parameter space as complete as possible, guaranteeing that the Pareto frontiers are found. *Finally,*

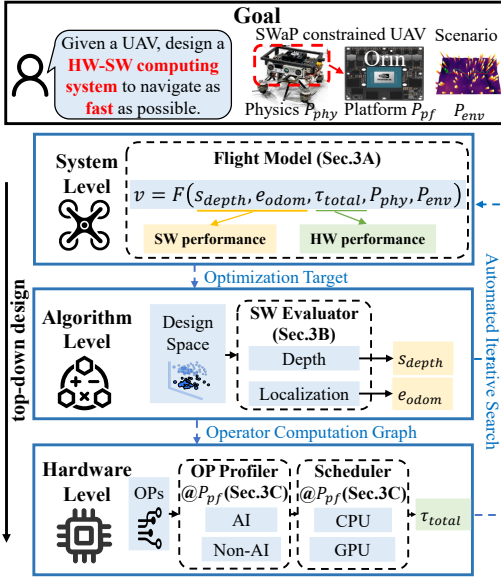


Figure 2: The overview of FastPilot. Our goal is to maximizing the navigation speed for a specific UAV with fixed physics P_{phy} and computing platform P_{pf} in a specific scenario P_{env} . We first model the relationship between the system-level metric and the SW-HW performance. Guided by the model, we traverse the design space and obtain the optimal design.

perception operators are complex, where the non-AI operators lack optimization and evaluation. AutoPilot [6] only considers end-to-end AI-based flight system and convolutional operators, which only support low-speed flight in simulation.

To solve these problems of existing design methods, this paper proposes a top-down automatic hardware-software design framework for high-speed UAVs, named FastPilot. We demonstrate the overview of this framework in Fig. 2 and summarize the contributions as follows:

- At the system level, we introduce a flight model that comprehensively build the relationship between system metric (flight speed) and the accuracy and latency of each perception module, thus supporting the joint optimization of multiple modules on algorithm and hardware level.
- At the algorithm level, we construct evaluation systems for UAV's perception algorithm accuracy, and then typical algorithm parameters are extracted as the design space based on the domain knowledge and corresponding experiments.
- At the hardware level, relevant non-AI operators are extracted, profiled and accelerated by CUDA. Moreover, a scheduling strategy is designed to minimize latency on embedded CPU/GPU heterogeneous computing platforms.

2 RELATED WORK

2.1 High-speed UAV Navigation

How to achieve agile flight of UAVs in unknown environments is a hot research topic in the field of robotics. As shown in Fig. 1, although a UAV navigation system consists of perception and decision parts, most current methods focus on designing efficient

planners to maximize flight speed. Ego-planner [14] adopts a map-free gradient-based planning framework to reduce computing time, and achieves 3m/s in wild scenarios. Further, some learning-based planners are proposed to generate more robust and aggressive trajectories, which achieves existing SOTA vision-based flight speed, 10m/s in most scenarios [8].

However, as these speeds approach the theoretical maximum [8], perception becomes the bottleneck. Falanga *et al.* [2] study the effects that sensing range and latency have on the maximum speed a robot can reach. Then Zhang *et al.* [13] takes the localization error into consideration. Intuitively, farther sensing range, more accurate localization, and shorter latency bring higher flight speed.

2.2 Design Automation for Robotic Systems

Robots are complex systems with tremendous heterogeneity from different aspects, leading to a huge design space. Hence, there is a need for automated hardware-software design methodologies.

Archytas [7] is a framework that automatically generates a synthesizable robot localization accelerator from the high-level algorithm description. However, this framework is only applicable to a single module and cannot automate the design of a larger system consisting of multiple modules. Hadidi *et al.* [3] quantify the design-space tradeoffs for the computational profile of drones, but they do not propose an automation method. AutoPilot [6] is the most similar work with ours. It automatically navigate the large design space and select a combination of navigation algorithm and hardware accelerator which meet the expected specification. Nevertheless, AutoPilot only supports automatic design for one type of algorithm (end-to-end navigation neural network) and operator (convolution), resulting in low-speed flight. Besides, hardware configurations and UAV's physics need modification to achieve AutoPilot's ideal performance, deviating from practical requirements, where the UAV body and onboard computing platform are fixed.

3 METHODS

3.1 System Level

Contrary to current bottom-up design methods that begin their optimization from a single and independent module, FastPilot targets the system-level metric which relies on a balance between not only the intra-module but also the inter-module accuracy and latency.

Therefore, we adopts the flight model proposed by Zhang *et al.* [13] as the start point of our top-down optimization. Unlike previous models [2, 6], this model completely considers the performance and latency of all parts in the perception layer. As shown in Fig. 2 and Table 1., the cross-product effect F across the depth estimation and localization modules on flight speed is quantified through this model. Moreover, the mutual constraints of algorithm accuracy performance and hardware computing performance in the model support us to conduct hardware-software co-design.

The detailed formula of the model can be referred in [13].

3.2 Algorithm Level

Guided by the flight model, we are able to jointly design and co-optimize the performance of depth estimation and localization at the algorithm level. We first use the domain knowledge about the UAV perception to summarize and extract the design space. For

Table 1: The symbols used in the flight model.

Symbol	Meaning	Level
F	functional relationship between variables	System
v	maximum safe flight speed (m/s)	System
s_{depth}	depth sensing range (m)	Algorithm
e_{odom}	localization error (%)	Algorithm
τ_{total}	total perception latency (s)	Hardware

Table 2: Perceptual algorithm parameters tuned in FastPilot.

	Hyper-parameter	Potential Values
Depth	# Channel	[8, 16, 32, 48, 64]
	# Layers	[3, 6]
VIO	# Frontend Type	[Shi-Tomasi, SP_Raw, SP_GhostNet, SP_GGghost, SP_MobileNet]
	# Window Size	[3, 6, 10, 15, 20]

UAV depth estimation, there are two typical types of algorithms, traditional and learning-based. With the rapid advancement of neural networks, learning-based methods are greatly superior to traditional ones such as Semi-Global Matching (SGM) on both accuracy and latency. Thus we only consider the learning-based methods and take StereoNet [5] as example. For localization, visual-inertial odometry (VIO) is the most popular algorithm. VIO consists of a frontend that extracts feature points on two adjacent frames and track them to output a local pose transformation, and a backend that optimizes several frames together for a more precise estimation. In this paper, we select VINS-Mono [9] as the representative localization algorithm.

There are diverse parameters in StereoNet and VINS-Mono. We run evaluation experiments to determine what parameters are influential and their corresponding reasonable values. For brevity, we list them in Table 2 without demonstrating the detailed experiments. For depth estimation, the number of channels and the number of refinement layers play a crucial role. For the VIO frontend, Shi-Tomasi [11] represents the traditional feature extraction method, while the remaining four represent SuperPoint [1] with different backbones. For the backend, we select the size of the sliding window as a more valid parameter, equal to the number of frames that are jointly optimized.

Finally, for each combination of parameters in the design space, we evaluate the accuracy of depth estimation and localization respectively. All evaluations are conducted on publicly available datasets. It is worth mentioning that the current popular evaluation metric for depth estimation is end-point error (EPE), not sensing range. Thus we derive it by measuring the absolute depth error at different ranges.

3.3 Hardware Level

Each combination of parameters selected in the algorithm level corresponds to a specific computation graph, which consists of various operators. For the neural network part, current machine learning tools, such as PyTorch and TensorRT, have defined, accelerated and profiled these AI operators well. In FastPilot, we mainly focus on the non-AI operators derived from the backend of VIO. We use CUDA to accelerate these critical non-AI operators and summarize them in Table 3. For each operator, we profile it on embedded CPU, and on embedded GPU if it can be implemented in CUDA.

Table 3: Accelerated operators used in FastPilot.

Operator	Meaning	Accelerating Method
C_{Jacob}	Calculate Jacobian matrix	parallel computation of thousands of Jacobian matrices
C_{Res}	Calculate optimization residual	parallel computation of thousands of residuals
CD	Cholesky decomposition	parallel computation of hundreds of chunks

Table 4: The parameters of UAV and test scenario used in the experiment.

Fixed UAV system (P_{phy} and P_{pf})			
max acceleration	max jerk	safe bound	computing platform
$20m/s^2$	$120m/s^3$	$0.35m$	Orin NX 16GB
Navigation Scenario (P_{env})			
average obstacle radius		average distance between obstacles	
$0.15m$		$2.5m$	

Currently, the Nvidia Jetson series is the most widely-used computing platform for UAVs, such as TX2, and Orin NX. All of them are heterogeneous CPU/GPU architectures. In order to maximize the use of computing resources, we design a scheduler to automatically allocate operators temporally and spatially. We formalize the problem as a flexible job-shop scheduling problem, and use a genetic algorithm to solve it.

After the profiling and scheduling, we get the total perception latency on a specific heterogeneous computing platform. Along with the algorithm performance, we can compute the safe flight speed for a HW-SW design according to the system model. Then the process repeats in a brute-force manner until we find the optimal design.

4 EXPERIMENTS

We evaluate FastPilot's co-design results for a micro UAV equipped with a Nvidia Jetson Orin NX. The physical characteristics P_{phy} and the settings of the target scenario P_{env} are listed in Table 4.

We select two manual and bottom-up design methods for comparison. The first one is the High Accuracy (HA) design. HA always selects the algorithm that can achieve best accuracy and then optimizes the corresponding computation flow on hardware for each module in UAV system. And the Low Latency (LL) design always prefer the most light-weight algorithm for each module to minimize its computing time.

For the design space shown in Table 2, we demonstrate the evaluation and search results of FastPilot in Fig. 3. FastPilot always select the design point marked in the green circle that maximize the flight speed. Since FastPilot traverse the design space automatically, HA and LL design are included in the search process, which are marked in blue and red respectively. The concrete parameters in these three designs are listed in Table 5.

Fig. 3(a) and (b) shows the results at system level. For the convenience of visualization, we combine the parameters from the 4 dimensions into 2 dimensions in Fig. 3(b). The black points represent the Pareto frontier designs, which can be derived from Fig. 3(a). We can find that, in addition to HA and LL designs, FastPilot design also belongs to the Pareto frontier. This is because that there is a

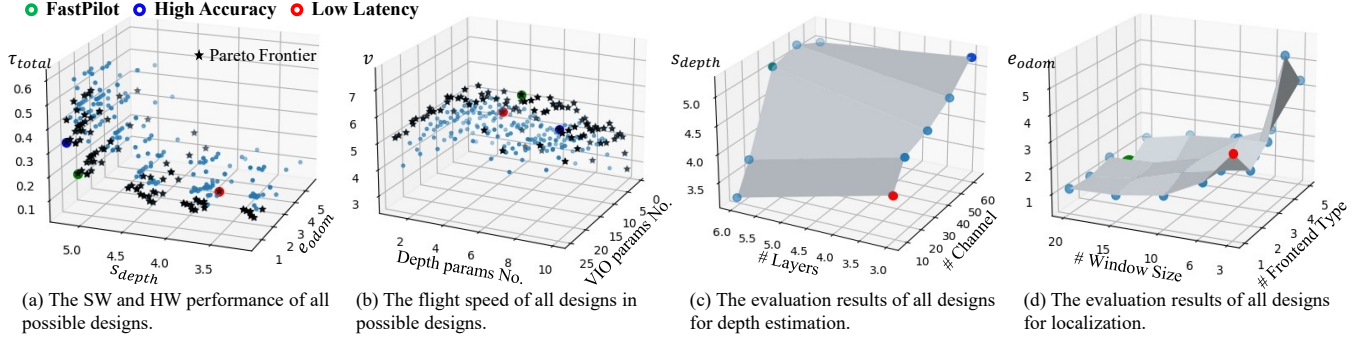


Figure 3: The evaluation results at different levels in FastPilot.

Table 5: The final parameters and system-level performance for the HA, LL and FastPilot's designs.

Design name	Channel	Layer	Frontend Type	Window Size	flight speed
HA	64	6	SP_GGghost	20	5.32m/s
LL	8	3	Shi-Tomasi	3	4.51m/s
FastPilot	32	6	SP_GGghost	20	7.46m/s

monotonic relationship between flight speed and the three SW-HW performance metrics according to the flight model in [13].

Fig. 3(c) and (d) shows the evaluation results at algorithm level, and can help us understand the choice of FastPilot design to some extent. Under the setting in Table 4, depth and localization accuracy have a more huge effect on speed than latency, which can be easily analysed using the tools in [13]. Therefore, FastPilot selects a combination of parameters that is similar with HA. However, from Fig. 3(c), we can notice that the growth trend of the sensing range slows down as the parameter increases, driving FastPilot to a half-way point with comparable sensing range and much less latency. Finally, FastPilot achieves 40% increase on flight speed than High Accuracy design, and 65% increase than Low Latency design.

In this section, we demonstrate FastPilot's ability to automate design in a particular case. Following the same search process, FastPilot is able to support different scenarios, different physics of UAV and computing platforms. Meanwhile, as a design framework, the libraries of algorithms and operators can be continuously updated with the development of UAV perception.

5 CONCLUSION & FUTURE WORK

FastPilot is a top-down automatic software-hardware design framework for high-speed UAV navigation. On the one hand, FastPilot presents new ideas for researchers in the field of robotics to improve the flight performance of UAVs from the SW-HW dimension; on the other hand, it provides researchers in the hardware field with an automated tool to tune their proposed designs in a complete robotics system. In the future, we will validate our design on a real drone and then replace the fixed embedded GPU with a reconfigurable FPGA to achieve fine-grained hardware optimization.

ACKNOWLEDGMENTS

This research was supported by National Natural Science Foundation of China (No.62203257, 62325405, U20A20334), Tsinghua University Initiative Scientific Research Program, Tsinghua-Meituan

Joint Institute for Digital Life, Beijing National Research Center for Information Science, Technology (BNRist) and Beijing Innovation Center for Future Chips.

REFERENCES

- [1] Daniel DeTone, Tomasz Malisiewicz, and Andrew Rabinovich. 2018. Superpoint: Self-supervised interest point detection and description. In *Proceedings of the IEEE conference on computer vision and pattern recognition workshops*. 224–236.
- [2] Davide Falanga, Suseong Kim, and Davide Scaramuzza. 2019. How fast is too fast? The role of perception latency in high-speed sense and avoid. *IEEE Robotics and Automation Letters* 4, 2 (2019), 1884–1891.
- [3] Ramyad Hadidi, Bahar Asgari, Sam Jijina, Adriana Amyette, Nima Shoghi, and Hyesoon Kim. 2021. Quantifying the design-space tradeoffs in autonomous drones. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 661–673.
- [4] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. 2023. Champion-level drone racing using deep reinforcement learning. *Nature* 620, 7976 (2023), 982–987.
- [5] Sameh Khamis, Sean Fanello, Christoph Rhemann, Adarsh Kowdle, Julien Valentin, and Shahram Izadi. 2018. Stereonet: Guided hierarchical refinement for real-time edge-aware depth prediction. In *Proceedings of the European Conference on Computer Vision (ECCV)*. 573–590.
- [6] Srivatsan Krishnan, Zishen Wan, Kshitij Bhardwaj, Paul Whatmough, Aleksandra Faust, Sabrina Neuman, Gu-Yeon Wei, David Brooks, and Vijay Janapa Reddi. 2022. Automatic Domain-Specific SoC Design for Autonomous Unmanned Aerial Vehicles. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 300–317.
- [7] Weizhuang Liu, Bo Yu, Yiming Gan, Qiang Liu, Jie Tang, Shaoshan Liu, and Yuhao Zhu. 2021. Archytas: A framework for synthesizing and dynamically optimizing accelerators for robotic localization. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 479–493.
- [8] Antonio Loquercio, Elia Kaufmann, René Ranftl, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. 2021. Learning high-speed flight in the wild. *Science Robotics* 6, 59 (2021), eabg5810.
- [9] Tong Qin, Peiliang Li, and Shaojie Shen. 2018. Vins-mono: A robust and versatile monocular visual-inertial state estimator. *IEEE Transactions on Robotics* 34, 4 (2018), 1004–1020.
- [10] Yunfan Ren, Fangcheng Zhu, Wenyi Liu, Zhepei Wang, Yi Lin, Fei Gao, and Fu Zhang. 2022. Bubble planner: Planning high-speed smooth quadrotor trajectories using receding corridors. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 6332–6339.
- [11] Jianbo Shi et al. 1994. Good features to track. In *1994 Proceedings of IEEE conference on computer vision and pattern recognition*. IEEE, 593–600.
- [12] Amr Suleiman, Zhengdong Zhang, Luca Carlone, Sertac Karaman, and Vivienne Sze. 2019. Navion: A 2-mw fully integrated real-time visual-inertial odometry accelerator for autonomous navigation of nano drones. *IEEE Journal of Solid-State Circuits* 54, 4 (2019), 1106–1119.
- [13] Suquan Zhang, Yuanfan Xu, Shu'ang Yu, Qingmin Liao, Jincheng Yu, and Yu Wang. 2024. Localization matters too: How localization error affects UAV flight. *arXiv preprint arXiv:2403.01428* (2024).
- [14] Xin Zhou, Zhepei Wang, Hongkai Ye, Chao Xu, and Fei Gao. 2020. Ego-planner: An esdf-free gradient-based local planner for quadrotors. *IEEE Robotics and Automation Letters* 6, 2 (2020), 478–485.